

Python Topic 6 — Tuples & Sets

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert

Layer 1 — New to Python

A **tuple** is a list that can't be changed — same idea as a list, but locked. Round brackets `()` instead of square `[]`.

```
point = (3, 5)           # a tuple
colors = ("red", "green", "blue")
point[0]                 # 3 → read by index, just like a list
```

Because it's locked, you can't add, remove, or change items:

```
point[0] = 10           # ❌ TypeError – tuples are immutable
```

A **set** is a bag of unique items — no duplicates, no order. Curly braces `{}`.

```
tags = {"python", "ai", "python"} # duplicate "python" disappears
print(tags)                       # {'python', 'ai'} → only unique kept
```

i Rule of thumb: use a **tuple** for fixed data (like coordinates), a **set** for unique items and fast "is this in there?" checks.

Layer 2 — Intermediate (real code + how it works)

Tuples

```
person = ("Akshay", 26, "Pune")
name, age, city = person           # unpacking → name="Akshay", age=26, city="Pune"

single = (42,)                   # ⚠️ one-item tuple NEEDS the trailing comma
not_a_tuple = (42)               # this is just the int 42!

# tuples often return multiple values from a function
def min_max(nums):
    return min(nums), max(nums)   # returns a tuple

low, high = min_max([3, 7, 1])   # low=1, high=7
```

Why tuples exist (vs lists)

- **Immutable** → safer; signals "this shouldn't change"
- **Faster** than lists (slightly)
- **Hashable** → usable as dict keys or set members (lists can't!)

```
locations = {(0, 0): "origin", (1, 2): "point A"} # tuple keys ✓
# {[0, 0]: "origin"} ✗ TypeError – list can't be a key
```

🔑 It's the Comma That Makes a Tuple — Not the Brackets

The single most common tuple confusion: parentheses usually just **group** (like in math). The **comma** is what creates a tuple.

```
a = ("akshay") # str → parens just group, no comma
b = ("akshay",) # tuple → the comma makes it a tuple ✓
c = "akshay", # tuple → works WITHOUT parens! comma is enough
```

Proof with a number:

```
x = (42) # int 42 – parens just wrap the number
x + 1 # 43 ✓ it's a number

y = (42,) # tuple containing 42
y[0] # 42 → it's a container now
```

Code	Type	Why
("akshay")	str	parens just group — no comma
("akshay",)	tuple	trailing comma → tuple
"akshay",	tuple	comma alone is enough
(1, 2, 3)	tuple	commas between items
()	tuple	empty tuple (special case)
(42)	int	grouping

i Two special cases: the empty tuple `()` needs no comma (nothing to disambiguate); everything else relies on commas.



Sets — deeper

```
nums = {1, 2, 3, 4}
nums.add(5)           # add an item
nums.discard(2)       # remove (no error if missing)
nums.remove(3)        # remove (errors if missing)
3 in nums             # membership check — VERY fast (O(1))

empty = set()         # ⚠️ {} makes an empty DICT, not a set!
```

Set math — the killer feature (Venn diagrams)

```
a = {1, 2, 3}
b = {3, 4, 5}

a | b      # {1,2,3,4,5}  union (all items)
a & b      # {3}          intersection (common items)
a - b      # {1, 2}       difference (in a, not b)
a ^ b      # {1,2,4,5}    symmetric difference (in one, not both)
```

✓ #1 real-world use — dedupe a list:

```
nums = [1, 2, 2, 3, 3, 3]
unique = list(set(nums))    # [1, 2, 3] → dedupe in one line
```



Layer 3 — Expert (internals & gotchas)

1. Why sets are O(1) for lookups — they're hash tables

```
x in my_list    # O(n) — scans every item
x in my_set     # O(1) — jumps straight to it via hash
```

Repeatedly checking "does this exist?" → convert to a set first. Big win for RAG/dedup work (seen document IDs, etc.):

```
seen = set()
for doc_id in stream:
    if doc_id in seen:    # O(1) each check
        continue
    seen.add(doc_id)
```

2. Sets/dict keys require hashable (immutable) items

```
{[1, 2], [3, 4]}      # ❌ TypeError: unhashable type 'list'
{(1, 2), (3, 4)}     # ✅ tuples are hashable
```

This is *why* tuples matter — they go where lists can't. Mutable things can't be hashed because their hash would change if they mutated.

3. Tuples are immutable — but can hold mutable things

```
t = (1, [2, 3])
t[1].append(4)      # ✅ works! → (1, [2, 3, 4])
t[1] = [9]          # ❌ TypeError – can't reassign the slot
```

⚠️ **Subtle:** the tuple's *structure* is locked (can't swap slot 1), but if a slot holds a mutable list, that list's contents can still change.

4. Sets are unordered — don't rely on order

```
{3, 1, 2}           # order never guaranteed; can't index: my_set[0] is TypeError
```

Need order + uniqueness? Use `dict.fromkeys()`:

```
list(dict.fromkeys([3, 1, 2, 1])) # [3, 1, 2] → dedupes AND keeps order
```

5. `frozenset` — an immutable set

```
fs = frozenset([1, 2, 3]) # can't change; can itself be a dict key / set member
```

Comparison table

Feature	List	Tuple	Set
Syntax	<code>[1, 2]</code>	<code>(1, 2)</code>	<code>{1, 2}</code>
Ordered	✅	✅	❌
Mutable	✅	❌	✅
Duplicates	✅	✅	❌
Indexable	✅	✅	❌

Hashable (usable as key)	✗	✓	✗ (frozenset ✓)
Lookup speed	O(n)	O(n)	O(1)

🎯 Interview Q&A Cards

Q: What makes something a tuple — the parentheses or the comma?

A: The comma. `("x")` is a string; `("x",)` and even `"x",` are tuples. Parens usually just group. Empty tuple `()` is the one exception.

Q: Why use a tuple instead of a list?

A: Immutability (safer, signals fixed data), slightly faster, and hashability — tuples can be dict keys or set members; lists cannot.

Q: Why are set lookups faster than list lookups?

A: Sets are hash tables → `x in set` is O(1). Lists scan linearly → `x in list` is O(n). Convert to a set for repeated membership checks.

Q: Can a tuple ever "change"?

A: Its slots can't be reassigned, but if a slot holds a mutable object (e.g. a list), that object's contents can still be mutated. The tuple structure is fixed; the referenced objects may not be.

🔗 **Memory hook:** *Tuple = locked list (round brackets, made by the comma). Set = unique bag, O(1) lookups, no order. Need a dict key? It must be immutable — reach for a tuple.*

✓ **One-liner for interviews:** "Tuples are immutable, ordered, and hashable — used for fixed data and as dict keys. Sets are unordered collections of unique, hashable items with O(1) membership — used for dedup and fast lookups. Both trade mutability for capabilities lists don't have."